

INSTANTLY SHAREABLE PLAYTEST

Implementation & Cross-Team Alignment Document
xPlaytest (Partner Center) × xCloud (Game Streaming Services)

Author	Manager	Audience	Status
Melanie Chen	Brian Bowman	xPlaytest + xCloud	In Review: May 29, 2026

Table of Contents

Table of Contents.....2

1. What We Are Building3

2. End-to-End Flow4

3. Team Responsibilities5

 3.1 xPlaytest6

 3.2 xCloud (Game Streaming Services — 4 Services)7

4. Data Contract — Trigger Payload7

 4.1 PlaytestIngestionJobParameters.....**Error! Bookmark not defined.**

 4.2 StoreAsset — Critical Fields.....9

5. Known Blockers10

6. Resolved Decisions11

7. Open Questions11

8. Success Criteria (P0)12

1. What We Are Building

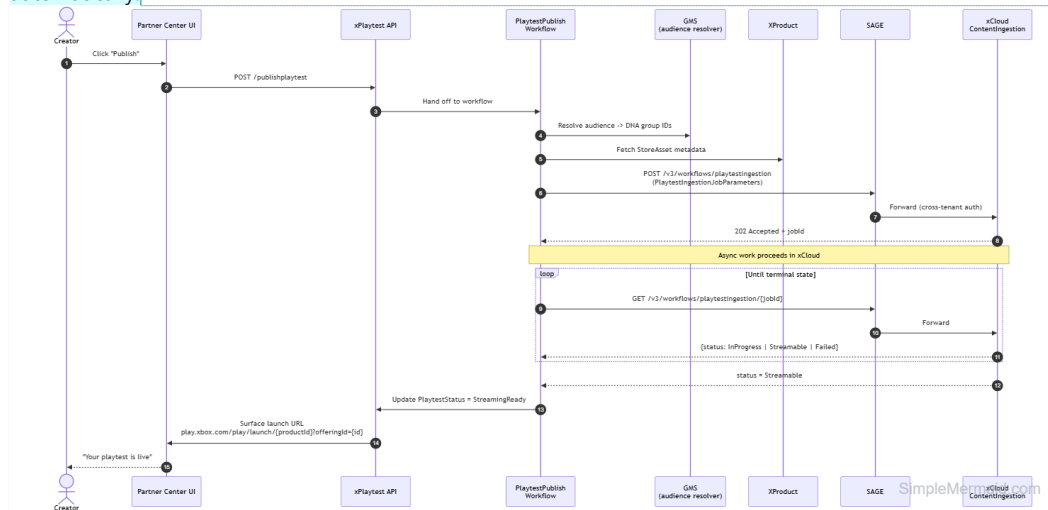
Today, **xPlaytest** allows game creators to share private pre-release builds with testers through **local installs** (Garrison). Because modern games can be hundreds of gigabytes, **downloads are slow**, hardware provisioning does not scale well, and distributing physical devices increases the risk of leaked unreleased code.

The Instantly Shareable Playtest project **wires xPlaytest directly into Xbox Cloud Gaming's** streaming pipeline. When a creator publishes a playtest with Cloud Streaming enabled, their (RETAIL-signed) PC build becomes streamable to authorized testers via a shareable link **with no install or download required**.

This is an **opt-in** feature: creators can still publish standard (downloadable) playtests, but if they check an **Enable Cloud Streaming** option at publish time, the Playtest's build will be ingested into xCloud for streaming **in addition to** being available for download. This approach leverages cloud capabilities to **speed up content access**, reduce device management overhead, and minimize exposure of game files.

2. End-to-End Flow

When a creator publishes a Playtest with cloud streaming enabled, the following four-phase sequence executes automatically:



Commented [B(1)]: Please add the diagrams directly into the doc where possible.

Commented [B(2)]: Is step 5 in this sequence diagram actually an XProduct fetch or do we just know the data from the Playtest document to call xcloud?

#	Phase	What Happens	Order
1	Publish Trigger	When a creator clicks Publish in Partner Center, the PublishPlaytest endpoint performs a quick validation to confirm the Playtest is ready. It then hands off to the existing PlaytestPublish workflow within XPackageWorkflow , where all publish-side processing takes place. This workflow already orchestrates cross-system operations (including calls to XProduct, SUCU, and GMS), making it the correct integration point for xCloud ingestion. It is also designed for long-running, retrieable execution, ensuring the downstream steps can complete reliably even in the presence of transient failures.	xPlaytest
2	xCloud Handoff	Within the PlaytestPublish workflow, xPlaytest reuses data it already resolves during the publishing process to construct a single PlaytestIngestionJobParameters payload. Currently, the publishing workflow: - resolves audience groups into DNA group identifiers via GMS - assembles product and package metadata that is written into XProduct The xCloud handoff packages these same outputs into one request by combining: StoreAsset: store-equivalent metadata derived from Playtest product/package data Audience and availability configuration: DNA groups, expiry, and sandbox This payload is sent to a SAGE (Service API Gateway) endpoint and routes the request to xCloud's content ingestion service , which processes it asynchronously. The request returns a 202 Accepted response with a jobId , indicating that the xCloud workflow has begun.	xPlaytest → xCloud
3	xCloud Ingestion Workflow	xCloud processes the request through a PlaytestTitleIngestionWorkflow , which runs entirely within xCloud and orchestrates the following steps: Ingest Build: Uses the provided StoreAsset to ingest the title and produce streaming-ready package identifiers. Create or Update Offering: Creates a Playtest-specific offering (xpt-playtestId) in Partner Registry and configures it with: - AllowedDnaGroups - expiration and sandbox settings Attach Title and Prepare Capacity: Attaches the title to the offering and waits for the install pipeline to provision infrastructure. This workflow has both ingestion and availability setup as a single coordinated workflow triggered by the Playtest ingestion request.	xCloud
4	Streamable	Once processing completes, xCloud signals a terminal state of Streamable via either a polling API or an asynchronous callback. xPlaytest consumes this signal, updates the Playtest status to StreamingReady , and surfaces a shareable, one-click launch URL (e.g. https://play.xbox.com/play/launch/{productId}?offeringId={id}) in the Partner Center UI for testers to use. If the xCloud ingestion ultimately fails or can't find an install, the signal will indicate a terminal Failed or InstallNotFound status so that xPlaytest can handle these cases gracefully.	xCloud → xPlaytest

Commented [EP3]: this is the current flow, we are looking for the confirmation from each team that this is correct

Commented [B(4)]: Do the row colors mean anything? Might be nice to highlight which phases are new / modified compared to the normal xPlaytest flows

Commented [MC5]: ResolveUserDnaGroupIds returns flightIds

Commented [B(6R5)]: Is this true? Wouldn't it be a DnaGroupId not an xbox flight id? Or is this yet another definition of 'flight id'?

Commented [MC7R5]: Yet another definition of flight id

Commented [B(8)]: Is the idea that is all happening in one REST call to ingestion or are we thinking that creating an offering and content ingestion will be separate calls?

Commented [MC9R8]: Yes. All of the offering and ingestion logic will happen in one workflow within xCloud. xPlaytest makes exactly one cross-tenant call (POST /playtest/ingestion via SAGE)

Commented [B(10)]: consider if we should have any easier / more human readable naming since these will be somewhat visible in the UX. I think the answer is probably no as we need to maintain uniqueness and it would be a pain to do other mappings but somewhere in your spec we should discuss the 'interesting design tradeoffs' being made / why contracts look the way they do

Commented [MC11]: OfferingsProcessor.AddOrReplaceOfferingAsync runs: Validate.HasLengthInRange(offering.Id, "OfferingId", 2, 32);

Commented [B(12)]: What does this actually mean / is there anything unique or needing to be changed for us with this? (Basically how is this different, if at all, from when a normal new game enters streaming)

Commented [MC13R12]: Unlike from how a normal new game goes into streaming, Playtest will supply the StoreAsset directly and does not rely on XProduct or BigCat lookups

Commented [B(14R12)]: Sure but that is basically just writing a database record that says 'xcloud knows about these games' ... preparing capacity has an implication that servers are being spun up / the game is being cached somewhere easier to access when a stream starts / etc. ... there is a lot of this flow that Anthony Popelar and the Streaming Server team know about that I think we can black box and say 'happens just like every other game' but I'm not sure if you were implying any other difference besides where the information is sourced from.

3. Team Responsibilities

3.1 xPlaytest Deliverables

ID	Task	Notes
PT1	Integrate xCloud trigger into PlaytestPublish workflow	Reuse existing workflow orchestration and retry behavior. If the xCloud ingestion fails, it does not block the overall Playtest publish (the Playtest will still publish for download even if streaming fails)
PT2	Construct StoreAsset from Playtest product/package data	At the point where the xCloud trigger is built, the Playtest publish workflow already has the required package-level IDs in PlaytestLifecycleState, populated during FetchContentIds and updated during PollContentSubmission.
PT3	Build offering configuration from audience data	Use resolved DNA group IDs (via GMS) to populate AllowedDnaGroups.
PT4	Send ingestion request via SAGE	POST PlaytestIngestionJobParameters; store returned jobId. Only cross-tenant call (202 Accepted + jobId).
PT5	Handle completion signal	Update status to StreamingReady and surface launch link. Poll vs callback TBD (Q4).
PT6	Handle lifecycle updates	Re-trigger the ingestion if the Playtest's key parameters change after publishing: e.g. audience groups updated, playtest expiration changed, build re-published, or playtest deleted. xCloud should reprocess accordingly. (These updates will typically involve sending a new ingestion request with updated parameters to adjust the offering's config or clean up streaming resources.)

Commented [B(15): Super duper ideally we get deliverables created in ADO to track the work needed. If we have it down to the task granularity that is amazing for things with a super well known list of items to complete (tasks should be < 5 days of work). For items that we want smaller follow up 1 pagers on, its fine to keep deliverable level and have just a single task of 'write the one pager' under it for now.

I can help on the ADO side of things too!

Commented [B(16): Where possible it would be helpful to have a column for a 'SWAG' (stupid wild *** guess) of the time you think it will take. This helps us baseline capacity and timelines

Commented [B(17): You put this in 'call sequence' order it seems like which is totally fine but it would be good to get a sense from you on what tasks you will tackle first / what is parallelizable / etc. so we can get a better idea of overall timeline and expectations

Commented [MC18]: launch URL builder must be net-new code

3.2 xCloud (Game Streaming Services — 4 Services)

ID	Service	Task	Notes
XC1	SAGE (Service API Gateway)	Add a Playtest ingestion endpoint and a status endpoint for polling.	SAGE is the ingress layer for the cross-tenant handoff; xPlaytest calls SAGE, and xCloud processes the request asynchronously behind it.
XC2	SAGE (Service API Gateway)	Allow the Playtest Service S2S identity on the new route.	This is required to enable the cross-tenant call from the Playtest Service (MSFTGreen Tenant) to xCloud (Corp but migrating to PME).
XC3	services.contentingestion	Implement a dedicated new PlaytestTitleIngestionWorkflow that accepts the full StoreAsset and orchestrates ingestion, offering creation or update, and title attachment.	Store is abstract in xCloud by design, but today the only active implementation is BigCat. For Playtest, the workflow can use store-style metadata supplied by xPlaytest/Product instead of requiring a new xProduct integration in the existing path.
XC4	services.partnerregistry	Extend offerings to support AllowedDnaGroups in the Partner Registry's offering model and storage (e.g. add AllowedDnaGroups to PlayerAuthorizationOptions).	This enables Playtest Offerings to declare their tester audience directly as DNA-group GUIDs.
XC5	User Login / Offering Auth path	Add a DNA-group check to the existing offering access path so xCloud verifies GsToken.DnaGroups against offering.AuthorizationOptions.AllowedDnaGroups before allowing a user to access the offering.	No new Sessions-side or Permissions-side title authorization changes are needed. For the Playtest scenario, if access to the offering is already restricted by DNA groups, the title/game-side permission check can be disabled so any user who can access the offering can play the configured title.
XC6	Install pipeline (cross-cutting)	For Playtest titles, treat one DNA group ID as the effective SUCU flight ID when xCloud needs to resolve the version/install path. Because Playtest DNA groups materialize in SUCU as separate "flights" but point to the same effective build version for this scenario, select one deterministic DNA group GUID and use it as the single flight ID passed into the install pipeline. Audit downstream install code for assumptions that a real GA flight must exist.	xCloud can "cheat" and just pick one of the flights because they all refer to the same version for the Playtest scenario. I will use the lexicographically smallest DNA group GUID as that deterministic default. This is an install-time lookup rule, not an audience-auth rule. XC5 controls offering access; XC6 only selects a single representative DNA group ID to use when SUCU expects one flight identifier.

Commented [MC19]: SAGE: [appsettings.xcloud.json](#)

implement 202 Accepted + jobId response contract

Commented [B(20R19)]: what is the appsettings json link here conveying? If its just a note to yourself thats fine but if its something you want readers to look at, we should have it in the doc proper

Commented [MC21R19]: SAGE's route-registration file. I linked it as a note to myself to add two JSON entries there

Commented [MC22]: CONTEXT: xCloud originally built the system so "store" could be a **generic concept** and potentially support things like BigCat, Steam, or other stores later, but they are still only using **BigCat** today. For my Playtest scenario, xCloud does **not** want to add a new direct xProduct integration right now; instead, since xPlaytest already writes to xProduct, it can also send xCloud the product metadata it needs "to the store.". So my new playtest workflow should therefore carry the needed **StoreAsset / store-style metadata in the workflow parameters**.

Commented [B(23)]: Personally would start with XC4 and XC5 items as they will not need other infrastructure to complete and then can be foundational features upon which you build the other pieces.

A huge step forward would be being able to manually create an offering in the portal, assign a DNA group id, and then go to partner center and adjust customer group membership and see it reflected in who can play games

Commented [MC24]: **PARTNER REGISTRY:** [OfferingV2.json](#)

Commented [JH25]: While we *could* make all the playtest offerings public and just gate the titles behind package flighting, we'll need auth service changes to support AllowedDnaGroups

Commented [B(26R25)]: This is an interesting design discussion. I *think* we will want to have playtest ...

Commented [JH27R25]: agreed!

Commented [B(28)]: This is kinda subtle but a really good design discussion to bring up with GSSV folks like @David Retterath / @Apars Walia ... I think it ...

Commented [JH29R28]: I think this should be fine (as long as offering access is sufficiently locked down). ...

Commented [MC30]: Playtest has **many DNA groups** for audience membership. SUCU/install logic still wants **one flight-like ID** when resolving the package version ...

Commented [MC31R30]: For Playtest package/version resolution in SUCU, use one Playtest DNA group as the effective flight identifier. Current ...

Commented [B(32R30)]: There is maybe an interesting discussion on what, if any, group the VM that installs the game before letting someone stream needs to be ...

Commented [MC33R30]: @Timi Bolaji could we walk through XC6? I want to make sure I have it correct, we pick a single representative DNA group GUID to hand ...

4. Data Contract — Trigger Payload

4.1 Purpose

When xPlaytest triggers xCloud ingestion, it should send a payload that contains:

1. identifiers xCloud needs to create the Playtest-specific **offering and title linkage**
2. **store-equivalent metadata** xCloud needs to ingest the build without introducing a new direct xProduct integration. In short, xCloud expects xPlaytest to provide all the data that xCloud would normally obtain from the store (BigCat) when ingesting a title. This matches the agreed approach that xPlaytest will supply xCloud with an XProduct-sourced **StoreAsset** object to represent the Playtest's build.

Table A: PlaytestIngestionJobParameters (xPlaytest builds and sends)

The request payload, sent by xPlaytest to xCloud (via SAGE API Gateway), is of type PlaytestIngestionJobParameters. It encodes the playtest's identity (for offering creation/updating), allowed testers, and relevant metadata. The fields and their origins are summarized below:

xCloud Field	Type	xPlaytest Source
PlaytestId	Guid	PublishedPlaytestEntity.PublishedPlaytestId This value is used by the xCloud ingestion workflow to derive the offering identifier (xpt-{id})
AllowedDnaGroups	string[]	PlaytestPublishJobParameters.FlightIds <i>(already resolved from GMS / DNA groups at publish time)</i>
ExpirationTime	DateTime	PlaytestPublishJobParameters.PlaytestEndDate
AllowedSandboxId	string	PackageConstants.RetailSandbox.
StoreAsset	object	StoreAsset is not persisted within Playtest. It is constructed at publish time from PlaytestLifecycleState (runtime/package identifiers) and PlaytestResponse (metadata), and then passed to xCloud as part of the ingestion payload.

Commented [MC34]: Every field in this mapping can be assembled inside the existing XPackagePlaytestPublishWorkflow, at the PlaytestProductCreation stage, reusing the same data that already gets sent to XProductAggregator:

```
{
  "xPlaytestPayload": {
    "productBigId":
      "PublishedPlaytestEntity.PartnerCenterProductId",
    "publisherId": "PublishedPlaytestEntity.SellerId",
    "playtestIdentityKey":
      "PublishedPlaytestEntity.PublishedPlaytestId",
    "allowedDnaGroups":
      "PlaytestPublishJobParameters.FlightIds",
    "playtestStartDate":
      "PlaytestPublishJobParameters.PlaytestStartDate",
  }
}
```

Commented [B(35): @Timi Bolaji / @Jack Heuberger to comment on this but its pretty 'route 1' to use a soccer term. This is totally fine if the desire is to build scenario specific workflows and REST endpoints (and then all the logic is straightforward but not reusable).

Commented [JH36R35]: Yeah this aligns with what Timi and I have talked about some with an orchestration layer on top of our workflow service, but that's definitely out of scope for now. We'd like to batch things as much as possible, but right now even our

Commented [MC37]: https://dev.azure.com/microsoft/Xbox/_git/Xbox.Xbet.Service?path=/src/PlayTest/PlayTest/BusinessLogic/PlaytestBusinessLogic.cs&version=GBT-melanichen/partner-registry-offering-

Commented [JH38]: Is this going to be user configurable? This should probably be locked down to a set maximum timeframe (e.g. 3 days)

Commented [B(39R38): @David Kushmerick do playtests have an expiration normally? I'd actually think the default would be much longer than 3 days

Commented [MC40R38]: PlaytestEndDate defaults to DateTime.MaxValue if the dev doesn't set an end date. Are long lived streaming offerings a storage constraint? @Jack Heuberger If they do, maybe a soft cap of 40

Commented [MC41]: Maps to TitleIngestion.JobParameters.Expiry

Commented [MC42]: Today PlaytestEndDate is user configurable with no upper bound and defaults to DateTime.MaxValue

Commented [B(43): If we are going a specific scenario route, consider removing any 'unused' or 'constant' values. It should be easy enough to have this default to RETAIL internally and not require callers to pass this.

Commented [MC44R43]: Note on what I know so far: POST /v3/workflows/playtestingestion & GET /v3/workflows/playtestingestion/(jobId) matches the WorkflowsControllerV3 pattern in services.contentingestion

Table B StoreAsset (xPlaytest fills)

xCloud Field	Type	xPlaytest Source
StoreEntry.ProductId	string	PlaytestPublishJobParameters.PlaytestGeneratedXProductBigId
StoreEntry.Flags	string	Constant "GAME"
StoreEntry.Name	string	PlaytestEntityBase.PlaytestName
StoreEntry.Description	string	PublishedPlaytestEntity.PlaytestDescription
StoreEntry.PublisherId	string	PlaytestEntityBase.SellerId
StoreEntry.IsPrivateAudience	bool	true
StoreEntry.ParentProductIds	string[]	[]
Markets	IReadOnlyCollection<Id>	["US"]
Platform	Id	PlaytestConstants.DefaultPlatform = "WINDOWS.DESKTOP"
ContentId	string	PlaytestLifecycleState.PlaytestContentId
XboxLiveTitleId	uint?	XProduct alternatelds[XboxTitleId]
PackageFamilyName	string	PlaytestLifecycleState.PackageFamilyName
AumID	string	Must source {PackageFamilyName}!{ApplicationId} where ApplicationId comes from XProduct

Commented [MC45]: StoreAsset(StoreEntry, Markets, Platform, ContentId, XboxTitleId?, PackageFamilyName?, AumID?)

Commented [B(46)]: Ignore my previous advice about removing constants if we are re-leveraging an existing data type though. This is fine here since StoreAsset is a known contract right?

Commented [MC47R46]: Yes, correct

Commented [JH48]: @Timi is the xcloud title id OK to be generated from this field?

Commented [TB49R48]: Yep the name is sufficient.

Commented [MC50]: Constant true all Playtests are private

Commented [MC51]: Playtests do not have a parent-product hierarchy in xPlaytest data. Send empty array.

Commented [B(52R51)]: @David Kushmerick would be good to confirm playtest's roadmap on supporting hub games / any other more complicated topologies

Commented [MC53]: PlaytestConstants.DefaultMarket = "US" and DefaultMarkets = [DefaultMarket] Playtest is single-market today; no multi-market XProduct lookup happens in the publish path.

Commented [MC54]: PlaytestProductDocument.ContainsMSIXVCByDeviceFamily is also hardcoded to {"WINDOWS.DESKTOP": true}

Commented [MC55]: Must equal wrapper sourceid

Commented [MC56]: AumID format is {PackageFamilyName}!{ApplicationId}. PackageFamilyName is available in PlaytestLifecycleState. ApplicationId is extracted from the AppX manifest during upload (AppxManifestCracker → PCPackageEntity.PackageInfo.Applications[*].Manifest.ApplicationId) but is not currently propagated into the Playtest publish data path. xCloud's StoreAsset validator rejects empty AumID on PC-GAME, so this must be plumbed through either by adding ApplicationId to PlaytestLifecycleState or by calling XPackageCore from the publish workflow.

Commented [MC57R56]: scope v1 to Xbox-only

Commented [B(58R56)]: We need to decide PC vs Xbox and I think that is mostly dependent on what playtest team can support during your internship. How complicated would it be to pass along aumid to the right place since it is already known in the upload process?

Table C: Derived inside **PlaytestTitleIngestionWorkflow** (xPlaytest does NOT send)

xCloud Field	Type	How it is computed
PartnerId	string	C - Microsoft
TitleId	string	"x (PlaytestEntityBase.PlaytestName + PlaytestGeneratedXProductBigId
SourceId	string	StoreAsset.ContentId
PackageNameOverride	string	StoreAsset.StoreEntry.Name as the display name for the offering.
Audiences	ICollection<XusAudience>	AllowedDnaGroups.Select(g => new XusAudience(g, XusAudience.RetailSandboxId)).
AssetId	Guid	Generated during asset ingestion; includes Packageld and ContentVersionId.
StreamingPackageIds	ICollection<Guid>	Retrieved from SUCU using SourceId and Audiences, including version and size information.

The new **PlaytestTitleIngestionWorkflow** is an orchestrator, not a reimplementation. It validates the inbound `PlaytestIngestionJobParameters`, wraps the contents into the shapes the existing ingestion contracts expect, and chains the existing `AssetIngestionWorkflow` and offering/title-attach steps as child stages. Because of this, the workflow only has to construct a small number of routing fields; everything else is read from the job state of the chained workflows.

The net-new code in `services.contentingestion` is the C.1 wrapping logic plus the orchestration glue between ingest → offering upsert → title attach. The workflow does not need to re-implement SUCU querying, asset creation, version ingestion, or streaming-package composition since those remain the responsibility of `AssetIngestionWorkflow` and the existing product/title ingestion code paths.

Commented [JH59]: quick callout -- xcloud will need some more fields configured for PC titles than what's listed here, but those won't change between titles

Commented [JH60R59]: The PC config space is not finalized at all and is guaranteed to change between public preview + launch when we get capacity

Commented [MC61]: This keeps Playtest data namespaced away from real xCloud partners and avoids requiring xPlaytest to know an xCloud-internal partner identity

Commented [MC62R61]: writes every Playtest offering under /Partners/PLAYTEST/Offerings/xpt-{shortId}

Commented [B(63R61): I dont think we want to segment this off actually. Its a great instinct and good callout but I dont think our systems are actually fully setup to handle anything that isn't in the Microsoft PartnerId anymore ... this is super legacy and I'd be wary of relying on it. For internship it seems fine to stick in Microsoft and deal with namespacing simply with the xpt- prefix

@Timi Bolaji though

Commented [TB64R61]: Yeah no need for PartnerId - the very nature of it means we don't want to expose it to partners.

Tbh my thinking was to just append the playtest productId to the TitleId. That'll give us uniqueness if a dev has multiple playtests, and allow them to share the same offering

Commented [B(65R61): sharing the same offering for other titles is ... complicated ... on the auth side of things. I'd like to keep it simple 1 playtest = 1 title = 1 offering for now ... we can use the client to do some fancy multi offering workflows if / when we get there (imagine logging into all/multiple of the private offerings you have access to and having those products exposed nicely in your library side by side with the 'prod' experience)

Commented [TB66]: We usually want our TitleId in xCloud to be user-readable, so we generate it by removing all whitespace & special characters from the game's name.

For uniqueness, we can append the PlayTest ProductId

Commented [MC67]: `StoreAsset.ContentId.AssetIngestion.JobParameters.Validate()` enforces `SourceId == StoreAsset.ContentId`, so the workflow must echo this through.

Commented [MC68]: Required whenever the titlecollection is set

Commented [MC69]: Required input To `AssetIngestionWorkflow.TriggerVersionIngestionJobs` ...

Commented [MC70]: Returned by `assetsProcessor.CreateAssetAsync(...)` inside `AssetIngestionWorkflow.CreateAssetAsync`

5. Known Blockers

Blocker	Problem	Solution
Cross-Tenant S2S	Playtest Service runs in the MSFTGreen tenant. xCloud only supports cross-tenant authorization between CORP and PME. There is no support between Green and PME.	Pull request 15715445: Adding CrossTenant Auth support for Corp->Green requests - Repos
Partner Registry PR Approvals	Every Partner Registry write today creates an ADO PR that SFI policy forbids the caller from self-approving. Our flow creates 2 PRs per publishing (offering write + title attach). Manual approval breaks the workflow.	Possible with a client method that consumes this PR: Pull request 15685697: Allow for the creation of mixed registry object PRs - Repos

6. Resolved Decisions

- Cloud does not need to read xProduct directly.
xPlaytest assembles the full StoreAsset and includes it in the trigger payload. xCloud consumes that payload through a Playtest-specific workflow path, so the existing BigCat-backed ingestion flow does not need to be changed.
- Offerings are created only when a Playtest is published.
The offering is created from within XPackageWorkflow, after PublishPlaytest has handed off to the PlaytestPublish workflow. This ensures offerings are created only when the creator intentionally publishes a Playtest, rather than on every package upload or intermediate package update (to prevent a lot of offerings).
- All Playtests use the RETAIL sandbox.
The current publish flow uses the retail sandbox as a constant. As a result, SandboxId is treated as a fixed value in the trigger payload and mapped into the offering configuration on the xCloud side.

7. Open Questions

- Terminal-state delivery contract is still open.
xPlaytest needs a reliable way to learn when the xCloud workflow has finished and whether it succeeded or failed. There are two viable options: xCloud can either expose a polling endpoint that xPlaytest calls with the returned jobId, or it can emit a callback / message that xPlaytest consumes asynchronously. The contract must distinguish, at minimum, between Streamable, Failed, and InstallNotFound so xPlaytest can surface the correct Playtest state in Partner Center.
- The Playtest identity key used to derive the xCloud offering still needs final alignment.
The current Playtest-side publish flow uses a published snapshot identity, while the xCloud draft assumes a stable Playtest-derived offering identity such as xpt-{id}. Before implementation, both teams need to agree on which identifier is the canonical key for offering creation and update behavior.

- Commented [B71]:** These updates do need to be reflected though right? If a publisher updates the playtest for a new nightly version of the game we will want the offering to be updated to have the latest version of the game available (not sure from a playtest perspective though if this is a new playtest or if one can be re-published / saved with new content.

Commented [MC72]: the offering id has to come from a key that's stable across republishes (the retail product id, or the draft ParentPlaytestId), not the per publishsnapshot id (PublishedPlaytestId) since it regenerates every time.

Commented [B73]: For any open Q, it would be great to have your recommendation on how this would be handled / closed

Commented [B74]: TBH this is probably easiest. Anything else will require some fancy EventHub type thing which will be fairly involved to setup

Commented [B75R74]: Interesting discussion though on how this fits into the xplaytest infra and how often to poll. some sort of back off / guestimate may help

Commented [MC76]: Polling?. xPlaytest hits GET /v3/workflows/playtestingestion/{jobId} on backoff until terminal state. WorkflowsControllerV3 already exposes job status in the existing shape so reuse this?

Commented [MC77]: xpt-{PartnerCenterProductId} not PublishedPlaytestId bec it rolls on every republish so every nightly build would create a new offering instead of updating the existing one

8. Success Criteria (P0)

- **Publishing a Playtest with Cloud Streaming enabled triggers xCloud ingestion automatically.**
When a creator publishes a Playtest — as opposed to saving a draft or uploading a package — the PlaytestPublish workflow should automatically trigger the xCloud ingestion request. This is the entry point for the end-to-end streaming flow.
- **A Playtest-specific offering is created and tied to the correct audience.**
After ingestion completes, xCloud should create an offering that is uniquely associated with the Playtest and gated to the intended tester audience. For v1, that audience is expected to be carried primarily as DNA-group IDs, with the offering enforcing access through the new AllowedDnaGroups path.
- **The offering contains exactly one title bound to the Playtest product.**
The xCloud workflow should attach a single title to the offering, and that title should correspond to the Playtest's product identity passed in the trigger payload. This confirms that the ingest → offer → attach sequence completed correctly.
- **Authorized testers can launch the Playtest through the cloud streaming link.**
Once the workflow reaches its terminal success state, authorized testers should be able to use the shareable launch URL and stream the Playtest through xCloud. This validates both the offering-access setup and the ingestion/install pipeline.
- **Audience and lifecycle changes propagate correctly after publish.**
If the Playtest audience changes, the expiry changes, the build is republished, or the Playtest is deleted, the integration should re-trigger the appropriate xCloud update path so that the offering stays in sync with the current Playtest state.
- **Unauthorized users do not learn anything about the Playtest from the launch link.**
If a non-member opens the launch link, the system should block access cleanly and avoid exposing Playtest metadata to users who are not part of the authorized audience. This is an important part of the security model for unreleased content.